

Основы функционального программирования

Практическая работа №2

Алгебраические типы данных и классы типов

Цель: Освоить применение `map`, `filter`, `foldr/l`, лямбда-выражений и композиции функций..

Теоретическая справка

- Сумма типов: `data Bool = True | False`
- Произведение типов: `data Point = Point Double Double`
- Рекурсивные типы: `data List a = Nil | Cons a (List a)`
- Синонимы типов: `type String = [Char]`
- Классы типов: `Eq`, `Ord`, `Show`, `Functor`, `Foldable`
- Deriving: автоматическое создание экземпляров

Пример решения

Задача: Определить тип `Complex` для комплексных чисел и объявить экземпляры `Num`, `Show`, `Eq`.

```
-- Определение типа
data Complex = Complex { real :: Double, imag :: Double }
    deriving (Eq)

-- Экземпляр Show: вывод в виде a+bi
instance Show Complex where
    show (Complex a b)
        | b >= 0    = show a ++ "+" ++ show b ++ "i"
        | otherwise = show a ++ show b ++ "i"

-- Экземпляр Num
instance Num Complex where
    (Complex a b) + (Complex c d) = Complex (a + c) (b + d)
    (Complex a b) * (Complex c d) = Complex (a*c - b*d) (a*d + b*c)
    abs (Complex a b) = Complex (sqrt (a*a + b*b)) 0
    signum (Complex a b) = Complex (a / r) (b / r)
        where r = sqrt (a*a + b*b)
    fromInteger n = Complex (fromInteger n) 0
    negate (Complex a b) = Complex (-a) (-b)
```

```
-- Дополнительная функция: модуль
magnitude :: Complex -> Double
magnitude (Complex a b) = sqrt (a*a + b*b)
```

Тестирование в GHCi

```
-- > let z1 = Complex 3 4
-- > let z2 = Complex 1 (-2)
-- > z1 + z2
-- 4.0+2.0i
-- > magnitude z1
-- 5.0
```

Ход работы

1. Создайте файл `Pr3_V<N>.hs`.
2. Определите тип данных согласно варианту.
3. Реализуйте все требуемые функции.
4. Объявите экземпляры классов типов вручную (без `deriving`, если не указано иное).
5. Протестируйте все операции.

Требования к отчёту

- Титульный лист (ФИО, группа, работа №1, вариант)
- Условие задачи
- Листинг кода с комментариями
- Диаграмма типа (как суммы/произведения)
- Листинг всех экземпляров классов типов с пояснениями
- Примеры использования всех функций
- Выводы: какие конструкции использовались, с какими трудностями столкнулись
- Формат PDF

Варианты заданий

Вариант 1. Арифметические выражения

Компонент	Описание
Тип <code>Expr</code>	Дерево выражений: • <code>Lit Int</code> — целочисленный литерал • <code>Add Expr Expr</code> — сложение • <code>Mul Expr Expr</code> — умножение • <code>Neg Expr</code> — унарный минус
<code>eval :: Expr -> Int</code>	Вычисляет значение выражения
<code>simplify :: Expr -> Expr</code>	Упрощает выражение: • $x + 0 = x, 0 + x = x$ • $x * 0 = 0, 0 * x = 0$ • $x * 1 = x, 1 * x = x$ • $\text{Neg (Lit } n) = \text{Lit } (-n)$
<code>instance Show Expr</code>	Выводит выражение в инфиксной форме со скобками

Вариант 2. Бинарное дерево

Компонент	Описание
Тип <code>Tree a</code>	Рекурсивный тип: • <code>Empty</code> — пустое дерево • <code>Node a (Tree a) (Tree a)</code> — узел с левым и правым поддеревьями
<code>height :: Tree a -> Int</code>	Высота дерева (количество уровней)
<code>toList :: Tree a -> [a]</code>	Обход дерева in-order (левое поддерево, корень, правое поддерево)
<code>fromList :: [a] -> Tree a</code>	Строит сбалансированное дерево из списка (без требования поиска)
<code>instance Foldable Tree</code>	Реализация <code>foldMap</code> или <code>foldr</code> для дерева

Вариант 3. Натуральные числа Пеано

Компонент	Описание
Тип <code>Nat</code>	Числа Пеано: • <code>Zero</code> — нуль • <code>Succ Nat</code> — следующее число
<code>toInteger :: Nat -> Integer</code>	Преобразование в целое число
<code>fromInteger :: Integer -> Nat</code>	Преобразование из целого числа

<code>instance Num Nat</code>	Арифметика без использования <code>Int</code> в реализации: • <code>(+)</code> , <code>(*)</code> , <code>(-)</code> — через рекурсию по <code>Succ</code> • <code>abs</code> , <code>signum</code> , <code>fromInteger</code>
<code>instance Eq Nat</code>	Равенство через структурное сравнение
<code>instance Ord Nat</code>	Порядок через рекурсию

Вариант 4. Многочлены

Компонент	Описание
Тип <code>Polynomial</code>	Новый тип-обёртка: <code>newtype Polynomial = Poly [Double]</code> Коэффициенты от старшего к младшему
<code>degree :: Polynomial -> Int</code>	Степень многочлена (наивысшая степень с ненулевым коэффициентом)
<code>evaluate :: Polynomial -> Double -> Double</code>	Вычисление значения в точке <code>x</code>
<code>add :: Polynomial -> Polynomial -> Polynomial</code>	Сложение многочленов
<code>multiply :: Polynomial -> Polynomial -> Polynomial</code>	Умножение многочленов
<code>instance Num Polynomial</code>	Арифметика через <code>add</code> , <code>multiply</code> , <code>negate</code>
<code>instance Show Polynomial</code>	Красивый вывод: <code>2x^2 + 3x + 1</code>

Вариант 5. Простой JSON

Компонент	Описание
Тип <code>JSONValue</code>	Рекурсивный тип: • <code>JNull</code> • <code>JBool Bool</code> • <code>JNumber Double</code> • <code>JString String</code> • <code>JArray [JSONValue]</code> • <code>JObject [(String, JSONValue)]</code>
<code>jsonToString :: JSONValue -> String</code>	Преобразование в строку формата JSON
<code>getString :: JSONValue -> Maybe String</code>	Безопасное извлечение строки
<code>instance Show JSONValue</code>	Вывод через <code>jsonToString</code>

Вариант 6. Список как АД

Компонент	Описание
Тип <code>LinkedList a</code>	АД списка: • <code>Nil</code> • <code>Cons a (LinkedList a)</code>
<code>reverse :: LinkedList a -> LinkedList a</code>	Разворот списка
<code>zipWith :: (a -> b -> c) -> LinkedList a -> LinkedList b -> LinkedList c</code>	Обобщённое слияние двух списков
<code>take :: Int -> LinkedList a -> LinkedList a</code>	Взять первые <code>n</code> элементов
<code>drop :: Int -> LinkedList a -> LinkedList a</code>	Пропустить первые <code>n</code> элементов
<code>instance Functor LinkedList</code>	Реализация <code>fmap</code>
<code>instance Foldable LinkedList</code>	Реализация <code>foldMap</code> или <code>foldr</code>

Вариант 7. Рациональные числа

Компонент	Описание
Тип <code>Rational</code>	<code>data Rational = R Int Int</code> — числитель и знаменатель
<code>reduce :: Rational -> Rational</code>	Сокращение дроби на НОД, нормализация знака (знаменатель > 0)
<code>add :: Rational -> Rational -> Rational</code>	Сложение: $a/b + c/d = (ad + bc) / bd$
<code>multiply :: Rational -> Rational -> Rational</code>	Умножение: $a/b * c/d = ac / bd$
<code>toDouble :: Rational -> Double</code>	Преобразование в <code>Double</code>
<code>instance Num Rational</code>	Арифметика через <code>add</code> , <code>multiply</code> , <code>negate</code>
<code>instance Fractional Rational</code>	Деление: $a/b / c/d = ad / bc$
<code>instance Eq Rational</code>	Равенство через сокращение
<code>instance Ord Rational</code>	Порядок через преобразование в <code>Double</code> или кросс-умножение

Вариант 8. Матрица

Компонент	Описание
Тип <code>Matrix a</code>	<pre>data Matrix a = Matrix { rows :: Int, cols :: Int, elems :: [a] }</pre> Элементы хранятся построчно
<code>transpose :: Matrix a -> Matrix a</code>	Транспонирование матрицы
<code>get :: Int -> Int -> Matrix a -> Maybe a</code>	Получить элемент по индексам (0-based), <code>Nothing</code> при выходе за границы
<code>set :: Int -> Int -> a -> Matrix a -> Maybe (Matrix a)</code>	Установить элемент по индексам, возвращает <code>Nothing</code> при выходе за границы
<code>multiply :: Num a => Matrix a -> Matrix a -> Maybe (Matrix a)</code>	Умножение матриц, <code>Nothing</code> при несовместимых размерах
<code>instance Show a => Show (Matrix a)</code>	Вывод построчно

Вариант 9. Конечный автомат

Компонент	Описание
Тип <code>Automaton st</code>	<pre>data Automaton st = Auto { states :: [st], alphabet :: [Char], delta :: st -> Char -> Maybe st, start :: st, accepting :: [st] }</pre>
<code>accepts :: Eq st => Automaton st -> String -> Bool</code>	Проверяет, принимает ли автомат строку

Вариант 10. Выражения с переменными

Компонент	Описание
Тип <code>ExprF</code>	Расширение <code>Expr</code> из варианта 1: • <code>Var String</code> — переменная
Тип <code>Env</code>	<code>type Env = [(String, Int)]</code> — окружение (список связываний)
<code>evalWithEnv :: ExprF -> Env -> Maybe Int</code>	Вычисление с окружением. Возвращает <code>Nothing</code> , если переменная не определена

Вариант 11. Граф

Компонент	Описание
Тип <code>Graph a</code>	<pre>data Graph a = Graph { vertices :: [a], edges :: [(a, a)] }</pre> Ориентированный граф
<code>neighbors :: Eq a => a -> Graph a -> [a]</code>	Все соседи вершины (куда ведут рёбра)
<code>hasPath :: Eq a => a -> a -> Graph a -> Bool</code>	Проверка существования пути между вершинами (рекурсивный DFS)
<code>isConnected :: Eq a => Graph a -> Bool</code>	Связен ли граф (для неориентированного графа — все пары достижимы)
<code>instance Show a => Show (Graph a)</code>	Вывод в формате <code>"Vertices: [...], Edges: [...]"</code>

Вариант 12. Дата

Компонент	Описание
Тип <code>Date</code>	<pre>data Date = Date { day :: Int, month :: Int, year :: Int }</pre>
<code>nextDay :: Date -> Date</code>	Следующий день с учётом високосных годов и длин месяцев
<code>isValid :: Date -> Bool</code>	Проверка корректности даты
<code>daysBetween :: Date -> Date -> Int</code>	Количество дней между двумя датами (абсолютное значение)
<code>instance Eq Date</code>	Равенство дат
<code>instance Ord Date</code>	Порядок дат (хронологический)

Вариант 13. Очередь

Компонент	Описание
Тип <code>Queue a</code>	<pre>data Queue a = Queue { front :: [a], back :: [a] }</pre> Два списка: <code>front</code> (голова в начале) и <code>back</code> (хвост в обратном порядке)
<code>empty :: Queue a</code>	Пустая очередь

<code>enqueue :: a -> Queue a -> Queue a</code>	Добавить элемент в конец очереди (в <code>back</code>)
<code>dequeue :: Queue a -> Maybe (a, Queue a)</code>	Извлечь элемент из начала очереди. Если <code>front</code> пуст — перевернуть <code>back</code> в <code>front</code> . Возвращает <code>Nothing</code> для пустой очереди.
<code>instance Foldable Queue</code>	Обход элементов в порядке очереди

Вариант 14. Разреженный вектор

Компонент	Описание
Тип <code>SparseVector a</code>	<code>data SparseVector a = SV { size :: Int, elems :: [(Int, a)] }</code> <code>size</code> — общая длина, <code>elems</code> — ненулевые элементы (индекс, значение)
<code>dot :: Num a => SparseVector a -> SparseVector a -> a</code>	Скалярное произведение (сумма произведений на совпадающих индексах)
<code>scale :: Num a => a -> SparseVector a -> SparseVector a</code>	Умножение всех элементов на скаляр
<code>toList :: Num a => SparseVector a -> [a]</code>	Преобразование в плотный список с нулями
<code>instance Num a => Num (SparseVector a)</code>	Арифметика (сложение, умножение на скаляр через <code>scale</code>)

Вариант 15. Выражения с let

Компонент	Описание
Тип <code>Expr</code>	Расширение <code>Expr</code> из варианта 1: • <code>Let String Expr Expr</code> — <code>let x = e1 in e2</code>
Тип <code>Env</code>	<code>type Env = [(String, Int)]</code>
<code>eval :: Expr -> Env -> Int</code>	Вычисление с окружением. <code>Let x e1 e2</code> — вычислить <code>e1</code> , добавить <code>x</code> в окружение, вычислить <code>e2</code>